STORAGE · HOME CLUSTER · K3S

Longhorn, in plain terms
replicated block storage for a GPU homelab

Longhorn turns the local disks scattered across your nodes into one pool of **replicated block storage**. Every volume gets kept as **2+ live copies on different machines**, so a workload survives a dead disk and can move to another node. This writeup teaches how that works — and, importantly, where your intuition about "duplicating model files everywhere" is right and where it isn't — grounded in the exact boxes you run.

Nodes a1 · a2 · a3 · x1 (amd64) · spark (arm64)

Today local-path (pinned) + hostPath model caches

Goal redundancy + mobility (bin-packing)

2
LIVE COPIES
A volume mirrored to 2 nodes (your recommended replica count)

1
MOUNT AT A TIME
A volume attaches to ONE node — copies are for safety, not sharing

Live
REPLICATION
Synchronous — every write hits all replicas at once

≠
BACKUP
Replication is not backup — that's a separate, external copy

The core mental model (read this first)

A Longhorn Volume = one virtual disk with three parts:

- **Engine** — a tiny process on the node where the volume is *currently mounted*. All reads/writes go through it.
- **Replicas** — full copies of the volume's data, each living on a *different* node's disk. Default 3; you'll use **2**.
- **Synchronous mirroring** — the engine writes every block to *all* replicas simultaneously. They're never "stale"; they're a live mirror.

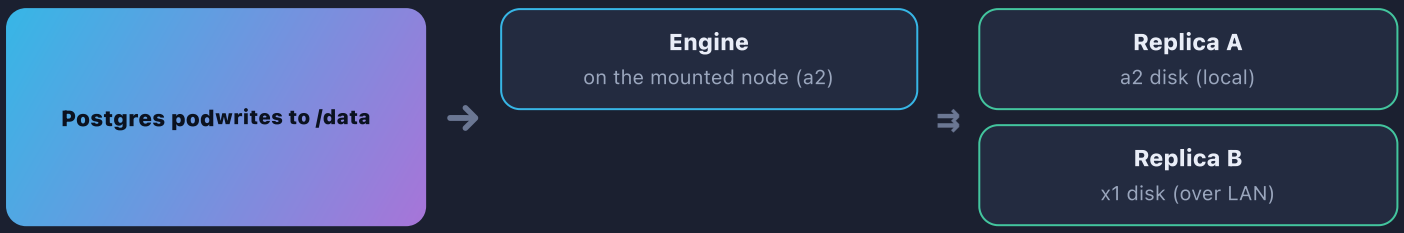
The one thing to unlearn

Replication is **not** "the model file is now readable locally on every computer." A replicated volume still mounts on **one node at a time** (ReadWriteOnce). The copies exist so the volume **survives a disk dying** and can **re-attach elsewhere** — not so two GPUs can read it at once. Duplicating a model to run it on both the Spark and a 4090 *simultaneously* is a different thing (covered on p.4).

HOW IT WORKS

Anatomy of a replicated volume

Say Postgres asks for a 20 GB volume with replica count 2. Longhorn picks two eligible nodes, lays down a replica on each disk, and starts an engine on whichever node the Postgres pod runs on:



One write → engine → **both** replicas at once. If a2's disk fails, the engine rebuilds from Replica B and the volume keeps serving. If the whole a2 node dies, Longhorn re-attaches the volume on a node that has a replica, and the pod reschedules there. **That** is the mobility that unlocks bin-packing.

Does it happen automatically? — Yes.

Automatic

- **Placement:** Longhorn's scheduler picks which nodes hold replicas (by free space + tags).
- **Mirroring:** every write is replicated live — you don't trigger it.
- **Healing:** a lost replica is rebuilt on a healthy node on its own.
- **Failover:** volume re-attaches to a surviving replica node automatically.

You control

- **Replica count** per volume (2 = durable; 1 = scratch, no redundancy).
- **Which nodes are eligible** via `node tags` / disable-scheduling.
- **Data locality** — keep a replica on the mounting node for local-speed reads.
- **Backups** — opt-in, per volume (p.5). Not automatic.

Your storage pool — who's in, who's out

NODE	ROLE FOR LONGHORN	WHY
a2 · 4090 · ~1.9 TB	`replica member`	Biggest disk, wired, amd64 — your storage anchor
x1 · no GPU · ~475 GB	`replica member`	GPU-free = no VRAM contention; ideal dedicated-ish storage (confirm it's wired)
a3 · 4090 · ~230 GB	`replica member`	Control plane; fine as a third replica home
a1 · 4090 · ~110 GB	`exclude`	**Flaky USB WiFi** — replication is network-heavy; it would corrupt/stall syncs
spark · GB10 · arm64	`exclude`	Different subnet (192.168.6.x) + arm64 + often offline — keep it pure compute

→ Longhorn on **a2 + x1 + a3**, default replica count **2**. Tag these `storage` and disable scheduling on a1 & spark.

MODEL FILES · THE CRUX

What replication does — and doesn't — do for model weights

Your instinct: "replication duplicates model files across computers so I can run models on different nodes." Here's the precise version.

Right

A model on a replicated Longhorn volume is **duplicated onto 2 nodes** and can **move between them without re-downloading**. Benchmark on a2 today, re-attach to a3 tomorrow — same bytes, no second 15 GB pull. Weight files (safetensors) are **just data**, so an amd64 and an arm64 node can both read the same bytes.

Wrong (the trap)

It does **not** make the file readable on *all* nodes at once. RWO = one mount at a time. And "can read the bytes" ≠ "can run the model": what a model can run on is decided by **the container image's architecture** (arm64 vs amd64) and **VRAM**, not by where the file sits. Replicating weights to a node that can't run the model just wastes disk.

Grounded in your fleet

CASE	RUNS WHERE	STORAGE CHOICE
trellis2 (arm64 image, huge)	Spark only	`hostPath on spark` — no other node can run it, so never replicate its data off-spark
Big model (only fits 128 GB)	Spark only	`hostPath / local on spark` — a 24 GB 4090 can't hold it; replicating elsewhere is dead weight
Mid model you shuffle between 4090s	a1/a2/a3	`Longhorn RWO, replica 2` — download once, re-attach to whichever 4090 you're testing
Scratch model you're just trying	anywhere	`Longhorn replica 1` or hostPath — don't pay 2× disk for a maybe-keeper

Three ways to hold a model file

hostPath cache `today`

Your `~/.cache/huggingface` per node. Fast, dead simple, node-local. No redundancy, no mobility — each node downloads its own copy. **Perfect for spark-only + re-downloadable weights.**

Longhorn RWO `recommended for 4090s`

One volume, 2 replicas, moves between the 4090s. Survives a disk loss, no re-download on move. **One node mounts it at a time.**

Longhorn RWX (ReadWriteMany) `avoid for big weights`

Longhorn can share one volume across nodes via an internal NFS layer — but it adds a network hop on every read, and streaming 15–100 GB of weights over cross-subnet NFS (to spark) is slow and fragile. Fine for small shared config; **not** for model loading.

WALKTHROUGH · ISSUE #7

You download Qwen3.6-27B (NVFP4) to benchmark on Spark and a 4090

This is exactly the scenario in Forgejo issue #7. Here's what happens, storage-wise, and the smart way to do it.

What you actually need: the file on two different-arch nodes at once

Spark (arm64, GB10) and a 4090 (amd64, Ada) can run *simultaneously* only if each has the weights locally. One RWO Longhorn volume can't mount on both at once — so replication alone doesn't cover "both at the same time." The clean play:

1. **Spark copy:** download into spark's `hostPath` HF cache (your current pattern). Spark isn't a Longhorn member anyway (arm64 / other subnet), so keep its weights node-local.
2. **4090 copy:** a Longhorn `RWO` volume, replica 2 on a2+x1, mounted to whichever 4090 you're testing. Download once; it survives a disk loss and re-attaches across 4090s for free.
3. **Result:** two copies total (one on spark, one Longhorn-backed for the 4090s) — the minimum for a true side-by-side. Longhorn didn't magically clone it to spark; you deliberately placed one copy per arch.

NVFP4 caveat worth logging in the benchmark

NVFP4 kernels generally want **Blackwell**. The **GB10 (spark) is Blackwell**; the **RTX 4090 is Ada** — it may fall back to a slower path or not run FP4 at all. So the storage question might be moot on the 4090 if the format won't execute there. (Captured as a checkbox in issue #7.)

Being smart about storage (your real worry)

Do

- Set **replica 1** for models you're just sampling; promote to 2 only for keepers.
- Keep **re-downloadable weights on hostPath** — don't back them up, don't 2×-replicate them.
- Right-size the PVC; Longhorn thin-provisions but replicas still cost real bytes ×N.
- Delete the volume when done — reclaims space on *all* replicas at once.

Don't

- Replicate a spark-only / arm64 model onto the 4090 nodes — pure waste.
- Put a 100 GB model on replica 3 "to be safe" — that's 300 GB gone.
- Use RWX to stream weights at load time.
- Treat a replica as a backup (delete-on-mount deletes all copies instantly).

Rule of thumb: **replicas × size = real disk used.** A 15 GB model at replica 2 = 30 GB across the pool. Budget models like you budget VRAM — pick what fits, evict what you're done with.

BACKUP VS REPLICATION

Untangling the "recursive backup" knot

Your worry: "Longhorn backs up to MinIO, but MinIO is itself replicated by Longhorn — that's recursive!" It feels that way because two different jobs share a name. They're not the same mechanism.

Replication

Live mirror, inside the cluster. N copies of a volume on N nodes, always in sync. Protects against a **dead disk / dead node**. Does **not** protect against "I deleted it" or "the cluster is gone" — those hit every replica at once.

Backup

Point-in-time snapshot, copied OUT to S3-style object storage. Incremental, versioned, separate from the live volume. Protects against **deletion, corruption, and disaster**.

Why it isn't actually recursive

- A backup is an **incremental blob of a snapshot** — inert data, not a live volume. Storing it doesn't "re-trigger" replication of anything.
- You simply **don't back a volume up into itself.** Longhorn backs up the volumes you *choose*; you exclude the backup target's own volume.
- **The real rule:** a backup you keep on the same disks it's meant to protect isn't a disaster backup. So the backup target should live **outside the cluster** — a cheap cloud bucket (Backblaze B2 / S3), a NAS, or a separate box.

Practical takeaway for your MinIO

Your in-cluster MinIO (`platform/minio-data`) is great as a Longhorn volume for *durability*, and fine as a backup target for *convenience* (undo a bad delete). But it is **not** a disaster backup — if the cluster burns down, so does it. For true safety, point Longhorn backups at an **offsite** S3 bucket. Then there's no loop: MinIO is replicated for uptime; the offsite bucket is the real backup.

Would Harbor / Forgejo / everything get backed up? How many times?

QUESTION	ANSWER
Does **every** volume get backed up?	No — only the ones you enable a backup schedule on. Backups are opt-in per volume (or per label group).
How many **replicas** (live copies)?	Whatever you set per volume — **2** for you. That's live redundancy, unrelated to backups.
How many **backup copies**?	Typically **1 target**, keeping the last *N* snapshots (retention). Incremental, so cheap.
Harbor registry (500 GB) / **Forgejo** (50 GB)?	Back these up — they're precious & hard to rebuild. Postgres, Vaultwarden, Immich, Paperless too.
Model weights / caches?	Don't back up — re-downloadable. Save the backup space for irreplaceable data.

The simple version (what to remember)

Three sentences

1. Replication keeps **2 live copies** of a volume on different nodes so it survives a dead disk and can move — it does **not** put your files on every machine at once. **2.** Backup is a **separate, external, point-in-time** copy for "oops" and "the cluster died" — keep it offsite, and it's not recursive because you never back a volume up into itself. **3.** You choose **per volume** what gets replicated and backed up — replicate/back up the irreplaceable (Harbor, Forgejo, DBs), keep re-downloadable model weights lean on hostPath.

Recommended rollout for this cluster

1. **Install Longhorn on a2 + x1 + a3, replica count 2**
   Tag them `storage`; disable scheduling on a1 (WiFi) and spark (arm64/subnet). Confirm `open-iscsi` is installed on the three members first.
2. **Migrate the irreplaceable volumes onto Longhorn**
   Forgejo (50 GB), Harbor registry (500 GB), Postgres, Vaultwarden, Immich, Paperless — one at a time, verify, keep local-path as a fallback until proven.
3. **Leave model caches on hostPath**
   Spark's HF cache and any spark-only / re-downloadable weights stay node-local. Use Longhorn RWO only for models you shuffle between 4090s.
4. **Add an offsite backup target, then schedule backups**
   Point Longhorn at a Backblaze B2 / external S3 bucket (not the in-cluster MinIO). Nightly backup + retention for the precious volumes only.

Concept cheat sheet

TERM	IN ONE LINE
Volume	A virtual disk your pod mounts (RWO = one node at a time).
Replica	A full live copy of a volume on one node's disk. You run 2.
Engine	The process on the mounted node that fans writes out to all replicas.
Data locality	Keep a replica on the mounting node so reads are local-speed.
Snapshot	In-cluster point-in-time marker (cheap, on the same disks).
Backup	A snapshot exported to external S3 — the real safety net.
Node tag	Label that controls which nodes may hold replicas.

Companion: Forgejo issue #7 (Qwen3.6-27B NVFP4 benchmark, Spark vs 4090). This doc lives at `docs/13-longhorn-replicated-storage.{html,pdf}`.